Design Rationale

TIVIDY is designed as a generic workflow management system, the design must support many different processes without being completely rewritten for each one. Our research identified recurring responsibilities (definitions, instances, work items, routing, assignment, events, history, external integration) and showed that these responsibilities vary independently in real systems. The design rationale explains why the system is structured the way it is, based on research and the proposed scope. The delivered implementation keeps every one of those responsibilities in its own interface or class, which is why the same engine executes both industries without a single industry-specific branch.

Separating Workflow Definitions From Running Instances

The separation between a workflow definition and a workflow instance established in the research phase of this project is the foundation of the design since it allows one definition to produce many independent instances without duplication. Each instance owns its own data, assignments, progress and history, so changes to one running instance cannot affect another. This is why the scope iterates that instances are created as independent executions, and why the information table separated definition-level information (activity descriptions, routing rules, required roles) from instance-level information (actual data values, assignments, progress, history)

Keeping Routing And Assignment Separate

Our research further showed that routing (which activity happens next) and assignment (who performs it) are distinct concerns, and that both are typically configurable rather than hard-coded. Our main example, Camunda, supports tasks assigned to specific users or made available to eligible users or groups, and organisations can change these policies without modifying the engine. The design keeps routing and assignment rules outside the basic work-item classes, so that policies can change independently of the work itself. Rules are changeable without rewriting the basic work-item classes.

Modelling The Work-Item Lifecycle Explicitly

The research also identified a recurring set of states that a work item passes through (e.g. created, available, assigned, started, completed, rejected, cancelled, escalated) and showed that operations valid in one state may not be valid in another. The design models the lifecycle explicitly, so that the engine can enforce valid transitions and prevent invalid actions. We include controlling valid actions and checking participant permissions. The lifecycle is not just a label, it determines what can happen next.

Handling Events And Triggers Separately From Core Work

Workflow progress can change when something happens (a task completes, a message arrives or a deadline passes). These are modelled as events. Multiple interested parts of a system may need to react to the same event, and external systems are often integrated through defined interfaces or connectors so that changes to external services do not

affect the core engine. The design separates event handling from the core workflow model, such that new rules, interested parties or integrations can be introduced without redesigning the engine. Event handling, external communication and failure handling are therefore separate responsibilities.

Supporting Added Behaviour Without Subclassing

A work item may acquire additional behaviour (e.g. validation, security which are delivered by decorator classes) without becoming a different kind of task. Auditing and notifications are handled at the instance-event level rather than per-item, which centralises them and avoids duplicating audit logic in every task class. Hard-coding these behaviours would make the design rigid and force duplication whenever a new combination is needed. We therefore allow behaviour to be added to work items independently of their core type, so that the same task can be validated, prioritised, audited or secured as needed.

Recording History Independently Of The Core Model

A workflow system must be able to explain its current progress and its past actions. History and audit records are essential for monitoring, reporting and recovery. The design treats history as a separate responsibility, recording actions, outcomes and state changes without removing earlier records. History is separated from the core work-item model and recovery actions are required to preserve the original history.

Control of Changes to Process Definitions

Changes to process definitions need control (e.g. Camunda normally keeps existing instances on the definition version they started with). We preserve the association in the instance's identity which records the version the instance was cloned from, and never changes. The library holds one active definition per name rather than a version history, a deferral made because the association may still be preserved, thus changing a definition never affects an instance already running.

Designing For Independent Variation

These decisions identify the parts of TIVIDY expected to vary independently (routing rules, assignment policies, approval and escalation logic, event handling, external integrations, behaviour attached to individual work items). The design keeps these separate from the core engine so that the system remains reusable across different processes. Our selected scenarios will, however, supply their own activities, data and policies, but will not determine the architecture of the engine itself.